```python
"""
╔═══════════════════════════════════════════════════════════════════════════╗
║  TERRA-IA — Pipeline complet v3.0                                        ║
║  Scoring de constructibilité morphologique — LiDAR HD IGN                ║
╚═══════════════════════════════════════════════════════════════════════════╝

CORRECTIONS V2 → V3
-------------------
  ✓ CRITIQUE : 24 dalles IGN (vs 4) → couverture ~80% commune Chambéry
  ✓ CRITIQUE : DVF Savoie depuis data.gouv.fr CSV direct (plus API Cerema)
  ✓ HAUTE    : Blocs spatiaux 300m (vs 200m) — meilleure indépendance CV
  ✓ HAUTE    : XGBRanker qid=block_id (groupes par contexte local, pas global)
  ✓ HAUTE    : GridSearch hyperparamètres XGBRanker
  ✓ MOYENNE  : Matrice de confusion par fold (diagnostic RF NDCG=1.000)
  ✓ MOYENNE  : Calibration τ softmin auto sur distribution CPI
  ✓ MOYENNE  : Analyse équilibre classes par bloc (blocs monoclasses détectés)
  ✓ BONUS    : svf_mean et aspect_south_ratio_mean ajoutés aux features ML
  ✓ BONUS    : Rapport statistique complet exporté en JSON

PRÉREQUIS
---------
    conda install -c conda-forge rasterio geopandas rasterstats numpy scipy richdem -y
    pip install scikit-learn xgboost shap rvt-py osmnx requests

USAGE
-----
    # Run complet depuis zéro
    python terra_ia_pipeline_v3.py

    # Skip téléchargement (données déjà présentes)
    $env:SKIP_DOWNLOAD="True"; python terra_ia_pipeline_v3.py          # Windows PS
    SKIP_DOWNLOAD=True python terra_ia_pipeline_v3.py                  # Linux/Mac

    # Skip téléchargement + recalcul features seulement
    $env:SKIP_DOWNLOAD="True"; $env:SKIP_FEATURES="True"; python terra_ia_pipeline_v3.py

RÉFÉRENCES SCIENTIFIQUES
------------------------
    Horn (1981)              Proceedings IEEE 69(1):14-47
    Riley et al. (1999)      Intermountain J. Sciences 5(1-4):23-27
    Beven & Kirkby (1979)    Hydrological Sciences Bulletin 24(1):43-69
    Barnes et al. (2014)     Computers & Geosciences 62:117-127
    Zevenbergen & Thorne (1987) Earth Surface Processes 12(1):47-56
    Zakšek et al. (2011)     Remote Sensing 3(2):398-415
    Ratner et al. (2017)     VLDB Journal 26:793-817   [Snorkel]
    Chen & Guestrin (2016)   KDD 2016                   [XGBoost]
    Burges et al. (2006)     ICML 2006                  [LambdaMART]
    Lundberg & Lee (2017)    NeurIPS 30                 [SHAP]
    Valavi et al. (2019)     Methods Ecol Evol 10(2):225-232
"""

# ── Stdlib
import os, sys, json, warnings, math
from pathlib import Path

# ── Scientific stack
import numpy as np
import pandas as pd
import geopandas as gpd
from scipy.ndimage import generic_filter, uniform_filter
from scipy.stats import spearmanr
import rasterio
import requests
from rasterio.merge import merge
from rasterstats import zonal_stats

warnings.filterwarnings("ignore")

# ============================================================================
# SECTION 0 — CONFIGURATION
# ============================================================================

DATA_DIR   = Path("data/lidar_chamberey")
RASTER_DIR = DATA_DIR / "rasters_v3"
```

```python
for d in [DATA_DIR, RASTER_DIR]:
    d.mkdir(parents=True, exist_ok=True)

# ────────────────────────────────────────────────────────────────────────────
# DALLES IGN V3 — Couverture étendue Chambéry
# V2 : 4 dalles (2×2 km centre-ville) → 16.4% parcelles valides
# V3 : 24 dalles (4×6 km)             → ~70-80% parcelles valides estimé
#
# Grille Lambert 93 :
#   X : 925000 → 929000 (4 colonnes × 1 km)
#   Y : 6499000 → 6505000 (6 lignes × 1 km)
#
# Pour ajouter une dalle : aller sur https://geoservices.ign.fr/lidarhd
# → Interface MNT ou MNH → Ctrl+clic sur la dalle → copier le lien
# ────────────────────────────────────────────────────────────────────────────

def _mnt_url(x: int, y: int) -> str:
    """Génère l'URL WMS IGN pour une dalle MNT de 1km² en Lambert 93."""
    return (
        f"https://data.geopf.fr/wms-r?SERVICE=WMS&VERSION=1.3.0&EXCEPTIONS=text/xml"
        f"&REQUEST=GetMap"
        f"&LAYERS=IGNF_LIDAR-HD_MNT_ELEVATION.ELEVATIONGRIDCOVERAGE.LAMB93"
        f"&FORMAT=image/geotiff&STYLES=&CRS=EPSG:2154"
        f"&BBOX={x-0.25},{y-0.25},{x+999.75},{y+999.75}"
        f"&WIDTH=2000&HEIGHT=2000"
        f"&FILENAME=LHD_FXX_{x//1000:04d}_{y//1000:04d}_MNT_O_0M50_LAMB93_IGN69.tif"
    )

def _mnh_url(x: int, y: int) -> str:
    """Génère l'URL WMS IGN pour une dalle MNH de 1km² en Lambert 93."""
    return (
        f"https://data.geopf.fr/wms-r?SERVICE=WMS&VERSION=1.3.0&EXCEPTIONS=text/xml"
        f"&REQUEST=GetMap"
        f"&LAYERS=IGNF_LIDAR-HD_MNH_ELEVATION.ELEVATIONGRIDCOVERAGE.LAMB93"
        f"&FORMAT=image/geotiff&STYLES=&CRS=EPSG:2154"
        f"&BBOX={x-0.25},{y-0.25},{x+999.75},{y+999.75}"
        f"&WIDTH=2000&HEIGHT=2000"
        f"&FILENAME=LHD_FXX_{x//1000:04d}_{y//1000:04d}_MNH_O_0M50_LAMB93_IGN69.tif"
    )

# Grille 4 colonnes × 6 lignes = 24 dalles MNT + 24 dalles MNH
_GRID_X = [925000, 926000, 927000, 928000]
_GRID_Y = [6499000, 6500000, 6501000, 6502000, 6503000, 6504000]

URLS_MNT = [_mnt_url(x, y) for y in _GRID_Y for x in _GRID_X]
URLS_MNH = [_mnh_url(x, y) for y in _GRID_Y for x in _GRID_X]

# Fichiers fusionnés
MNT_PATH       = DATA_DIR / "mnt_chamberey_v3.tif"
MNH_PATH       = DATA_DIR / "mnh_chamberey_v3.tif"
PARCELLES_PATH = DATA_DIR / "parcelles_73065.geojson"
DVF_PATH       = DATA_DIR / "dvf_73_2023.csv"

# ── Paramètres commune
COMMUNE_CODE   = "73065"
TARGET_CRS     = "EPSG:2154"
LAT_DEG        = 45.57   # Latitude Chambéry pour hillshade

# ── Fichiers de sortie
OUTPUT_CSV_V3   = "features_parcelles_v3.csv"
OUTPUT_CSV_ML   = "ml_dataset_v3.csv"
OUTPUT_REPORT   = "rapport_stats_v3.json"

# ── Flags
SKIP_DOWNLOAD  = os.environ.get("SKIP_DOWNLOAD",  "False").lower() == "true"
SKIP_FEATURES  = os.environ.get("SKIP_FEATURES",  "False").lower() == "true"

# ── Paramètres filtrage
SEUIL_NAN      = 0.40    # V3 : assoupli à 40% (vs 50% V2) pour plus de parcelles
SEUIL_SURFACE  = 5000.0  # m² max résidentiel
SEUIL_SURF_MIN = 20.0    # m² min

# ── Paramètres score CPI V3
TAU_SOFTMIN    = None    # Auto-calibré sur distribution (voir calibrate_tau)
```

```python
BETA_MIN_DEG   = 0.5     # Seuil anti-singularité TWI (Beven & Kirkby)

# ── Paramètres spatial CV V3
GRID_SIZE_M    = 300     # V3 : 300m (vs 200m V2) — meilleure indépendance
N_FOLDS        = 5

# ── Features organisées par groupe (pour CPI et ML)
FEATURE_GROUPS = {
    "PENTE":           ["slope_p50", "slope_p90", "slope_std"],
    "HYDROLOGIE":      ["twi_mean", "has_thalweg_mean"],
    "MORPHOLOGIE":     ["profile_curvature_mean", "tri_mean"],
    "ENSOLEILLEMENT": ["hillshade_winter_mean", "aspect_south_ratio_mean", "svf_mean"],
}
ALL_FEATURES = [f for g in FEATURE_GROUPS.values() for f in g]

# Poids groupes (spec collègue, validés littérature)
GROUP_WEIGHTS = {"PENTE": 0.35, "HYDROLOGIE": 0.30, "MORPHOLOGIE": 0.20, "ENSOLEILLEMENT": 0.15}


# ==============================================================================
# SECTION 1 — TÉLÉCHARGEMENT
# ==============================================================================

def download_file(url: str, dest: Path, label: str = "") -> bool:
    """Télécharge un fichier. Skip si déjà présent."""
    if dest.exists():
        size_mb = dest.stat().st_size / 1e6
        print(f"  ✓ Déjà présent : {dest.name} ({size_mb:.1f} Mo)")
        return True
    print(f"  ↓ {label} → {dest.name} ...", end=" ", flush=True)
    try:
        r = requests.get(url, stream=True, timeout=180)
        r.raise_for_status()
        with open(dest, "wb") as f:
            for chunk in r.iter_content(8192):
                f.write(chunk)
        print(f"OK ({dest.stat().st_size / 1e6:.1f} Mo)")
        return True
    except Exception as e:
        print(f"ERREUR — {e}")
        if dest.exists():
            dest.unlink()
        return False


def download_all_dalles(urls: list, prefix: str) -> list:
    """Télécharge toutes les dalles. Extrait le nom depuis FILENAME= dans l'URL."""
    print(f"  Téléchargement {len(urls)} dalles {prefix}...")
    paths, ok, skip, err = [], 0, 0, 0
    for i, url in enumerate(urls):
        fname = url.split("FILENAME=")[-1].split("&")[0] if "FILENAME=" in url \
                else f"{prefix}_dalle_{i:02d}.tif"
        dest = DATA_DIR / fname
        if dest.exists():
            skip += 1
            paths.append(dest)
        else:
            if download_file(url, dest, f"{prefix} {i+1}/{len(urls)}"):
                ok += 1
                paths.append(dest)
            else:
                err += 1
    print(f"  {prefix} : {ok} téléchargés / {skip} existants / {err} erreurs")
    return paths


def download_parcelles(code: str, dest: Path) -> bool:
    """Télécharge les parcelles cadastrales depuis cadastre.data.gouv.fr."""
    if dest.exists():
        print(f"  ✓ Parcelles présentes : {dest.name}")
        return True
    url = (f"https://cadastre.data.gouv.fr/bundler/cadastre-etalab"
           f"/communes/{code}/geojson/parcelles")
    print(f"  ↓ Parcelles {code} ...", end=" ", flush=True)
```

```python
    try:
        r = requests.get(url, timeout=120)
        r.raise_for_status()
        dest.write_bytes(r.content)
        print("OK")
        return True
    except Exception as e:
        print(f"ERREUR — {e}")
        return False


def download_dvf_v3(dest: Path) -> bool:
    """
    Télécharge les DVF (Demandes de Valeur Foncière) Savoie depuis data.gouv.fr.

    Source : https://files.data.gouv.fr/geo-dvf/latest/csv/
    Contient toutes les transactions immobilières du département 73.
    Licence : Etalab 2.0 — données open data depuis 2014.

    Les fichiers sont en gzip (.csv.gz) — pandas les lit nativement.

    Ref : Ministère de l'Économie et des Finances (2023).
          Demandes de valeur foncière (DVF).
          data.gouv.fr, dataset #5c4ae55a634f4117716d5656
    """
    if dest.exists():
        print(f"  ✓ DVF présent : {dest.name}")
        return True

    # Essai années en ordre décroissant — on prend la plus récente disponible
    years = ["2023", "2022", "2021", "2020"]
    for year in years:
        url = f"https://files.data.gouv.fr/geo-dvf/latest/csv/{year}/departements/73.csv.gz"
        dest_gz = DATA_DIR / f"dvf_73_{year}.csv.gz"
        print(f"  ↓ DVF Savoie {year} ...", end=" ", flush=True)
        try:
            r = requests.get(url, stream=True, timeout=180)
            r.raise_for_status()
            with open(dest_gz, "wb") as f:
                for chunk in r.iter_content(65536):
                    f.write(chunk)
            # Décompresser et filtrer sur la commune 73065
            df_raw = pd.read_csv(dest_gz, low_memory=False, compression="gzip")
            print(f"OK ({len(df_raw)} transactions dept 73)")
            # Filtrer commune
            mask = df_raw.get("code_commune", pd.Series(dtype=str)) == COMMUNE_CODE
            df_commune = df_raw[mask] if mask.any() else df_raw
            df_commune.to_csv(dest, index=False)
            dest_gz.unlink()  # Nettoyer le gz
            n = len(df_commune)
            print(f"    {n} transactions pour commune {COMMUNE_CODE}")
            return True
        except Exception as e:
            print(f"ERREUR — {e}")
            if dest_gz.exists():
                dest_gz.unlink()

    print("  ⚠  DVF : aucune année disponible — labels H3/H4 désactivés")
    return False


def merge_dalles(paths: list, output: Path, label: str) -> bool:
    """Fusionne des dalles GeoTIFF. Gère les dalles manquantes gracieusement."""
    valid_paths = [p for p in paths if p.exists()]
    if not valid_paths:
        return False
    if output.exists():
        print(f"  ✓ Fusion déjà faite : {output.name}")
        return True
    if len(valid_paths) == 1:
        import shutil
        shutil.copy(valid_paths[0], output)
        print(f"  ✓ Une dalle {label} — copiée.")
        return True
```

```python
    print(f"  ⚙  Fusion {len(valid_paths)} dalles {label} ...", end=" ", flush=True)
    try:
        datasets = [rasterio.open(p) for p in valid_paths]
        mosaic, transform = merge(datasets)
        meta = datasets[0].meta.copy()
        meta.update({"height": mosaic.shape[1], "width": mosaic.shape[2],
                     "transform": transform, "compress": "lzw",
                     "nodata": -9999.0})
        with rasterio.open(output, "w", **meta) as dst:
            dst.write(mosaic)
        [d.close() for d in datasets]
        size_mb = output.stat().st_size / 1e6
        print(f"OK ({size_mb:.1f} Mo, {len(valid_paths)} dalles)")
        return True
    except Exception as e:
        print(f"ERREUR — {e}")
        return False


# ==============================================================================
# SECTION 2 — VALIDATION DES DONNÉES
# ==============================================================================

def validate_raster(path: Path, label: str) -> bool:
    """Valide CRS, résolution et intégrité d'un raster IGN."""
    print(f"\n  Validation {label} : {path.name}")
    try:
        with rasterio.open(path) as src:
            crs_wkt  = str(src.crs)
            crs_epsg = src.crs.to_epsg()
            # IGN WMS encode EPSG:2154 sans authority — on vérifie les paramètres
            crs_ok = (str(crs_epsg) == "2154") or (
                "Lambert_Conformal_Conic" in crs_wkt and
                "700000" in crs_wkt and "6600000" in crs_wkt)
            res_ok = abs(src.res[0] - 0.5) < 0.01
            km_w   = src.width * 0.5 / 1000
            km_h   = src.height * 0.5 / 1000
            print(f"    CRS     : Lambert 93 / EPSG:2154  {'✓' if crs_ok else '✗'}")
            print(f"    Résol.  : {src.res}  {'✓' if res_ok else '✗ pas 50cm'}")
            print(f"    Taille  : {src.width:,} × {src.height:,} px  "
                  f"({km_w:.1f} × {km_h:.1f} km = {km_w*km_h:.1f} km²)")
            print(f"    Emprise : {src.bounds.left:.0f}, {src.bounds.bottom:.0f}"
                  f" → {src.bounds.right:.0f}, {src.bounds.top:.0f}")
            print(f"    NoData  : {src.nodata}")
        return True
    except Exception as e:
        print(f"    ✗ Erreur : {e}")
        return False


def load_parcelles(path: Path) -> gpd.GeoDataFrame:
    """Charge les parcelles cadastrales et les reprojette en Lambert 93."""
    print(f"\n  Chargement parcelles : {path.name}")
    gdf = gpd.read_file(path)
    print(f"    {len(gdf)} parcelles — CRS: {gdf.crs}")
    if gdf.crs is None or str(gdf.crs.to_epsg()) != "2154":
        gdf = gdf.to_crs("EPSG:2154")
        print("    Reprojection → EPSG:2154 ✓")
    valid = gdf[gdf.geometry.is_valid & ~gdf.geometry.is_empty].reset_index(drop=True)
    print(f"    {len(valid)} parcelles valides géométriquement")
    return valid


# ==============================================================================
# SECTION 3 — CALCUL DES FEATURES RASTER
# ==============================================================================

def compute_slope_raster(mnt_data: np.ndarray,
                         cellsize: float = 0.5) -> np.ndarray:
    """
    Pente pixel par pixel — méthode Horn (1981).
    Ref : Horn, B.K.P. (1981). Hill shading and the reflectance map.
          Proceedings of the IEEE, 69(1):14-47.
```

```python
    Formule : slope = atan(sqrt((dz/dx)² + (dz/dy)²)) × 180/π
    Fenêtre 3×3, pondération 1-2-1, diviseur = 8 × cellsize.
    """
    def horn(w):
        if np.any(np.isnan(w)):
            return np.nan
        z = w.reshape(3, 3)
        dzdx = ((z[0,2]+2*z[1,2]+z[2,2]) - (z[0,0]+2*z[1,0]+z[2,0])) / (8*cellsize)
        dzdy = ((z[2,0]+2*z[2,1]+z[2,2]) - (z[0,0]+2*z[0,1]+z[0,2])) / (8*cellsize)
        return np.degrees(np.arctan(np.sqrt(dzdx**2 + dzdy**2)))

    return generic_filter(
        mnt_data.astype(float), horn, size=3, mode="nearest"
    ).astype(np.float32)


def compute_tri_riley(mnt_data: np.ndarray) -> np.ndarray:
    """
    Terrain Ruggedness Index — Riley et al. (1999).
    Ref : Riley, S.J., DeGloria, S.D., Elliot, R. (1999).
          Intermountain Journal of Sciences, 5(1-4):23-27.
    Formule : TRI = sqrt(Σᵢ(eᵢ - e₀)²) pour les 8 voisins.
    """
    def tri(w):
        if np.any(np.isnan(w)):
            return np.nan
        return float(np.sqrt(np.sum((np.delete(w, 4) - w[4])**2)))

    return generic_filter(
        mnt_data.astype(float), tri, size=3, mode="nearest"
    ).astype(np.float32)


def compute_twi_and_thalweg(mnt_path: Path,
                            beta_min_deg: float = 0.5,
                            thalweg_cells: int = 500
                            ) -> tuple:
    """
    TWI urbain + masque thalweg en un seul passage D8 (optimisation V3).

    TWI (Beven & Kirkby 1979) :
        TWI_urban = ln(α / tan(max(β, β_min)))
        α = accumulation × cellsize²
        β = pente locale en radians
        β_min = 0.5° → évite singularité tan(0) en milieu plat urbain

    Thalweg : pixels où accumulation D8 ≥ seuil (500 cellules × 0.25m² = 125m² bassin)

    Amélioration V3 vs V2 :
        - Un seul calcul D8 (vs deux en V2)
        - Utilisation directe de rd.FlowAccumulation sans FlowDirectionD8 (API actuelle)
        - float64 pour accum → évite dépassement capacity sur grandes zones

    Ref pit-filling : Barnes et al. (2014). Computers & Geosciences 62:117-127.
    """
    try:
        import richdem as rd

        dem = rd.LoadGDAL(str(mnt_path), no_data=-9999.0)
        rd.FillDepressions(dem, epsilon=True, in_place=True)

        # Accumulation D8 — API richdem actuelle (FlowDirectionD8 retiré)
        accum_rd = rd.FlowAccumulation(dem, method="D8")
        accum    = np.array(accum_rd).astype(np.float64)  # float64 → plus de dépassement

        with rasterio.open(mnt_path) as src:
            mnt_arr  = src.read(1).astype(float)
            cellsize = src.res[0]
            nd       = src.nodata or -9999.0
        mnt_arr[mnt_arr == nd] = np.nan

        # Pente en radians
        slope_deg = compute_slope_raster(mnt_arr, cellsize)
        slope_rad = np.deg2rad(slope_deg.astype(float))
```

```python
        # TWI avec seuil anti-singularité
        beta_min_rad = np.deg2rad(beta_min_deg)
        slope_eff    = np.maximum(slope_rad, beta_min_rad)
        area         = accum * (cellsize ** 2)
        area         = np.maximum(area, cellsize ** 2)
        twi          = np.log(area / np.tan(slope_eff))
        twi          = np.clip(twi, None, 20.0)   # plafonnement urbain
        twi[np.isnan(mnt_arr)] = np.nan

        # Thalweg
        thalweg_mask = (accum >= thalweg_cells).astype(np.float32)
        thalweg_mask[np.isnan(mnt_arr)] = np.nan

        print(f"OK  TWI moy={np.nanmean(twi):.2f}  "
              f"thalweg={np.nansum(thalweg_mask==1):.0f} pixels "
              f"({np.nanmean(thalweg_mask==1)*100:.1f}% raster)")

        return twi.astype(np.float32), thalweg_mask

    except ImportError:
        print("⚠  richdem non disponible — TWI et thalweg ignorés")
        print("   conda install -c conda-forge richdem -y")
        return None, None
    except Exception as e:
        print(f"⚠  TWI/thalweg : {e}")
        return None, None


def compute_svf(mnh_path: Path) -> np.ndarray | None:
    """
    Sky View Factor — Zakšek, Oštir & Kokalj (2011).
    Ref : Remote Sensing, 3(2):398-415.
    Lib : rvt-py (EarthObservation/RVT_py, GitHub)
    Formule : SVF = (1/n) Σᵢ cos²(γᵢ)  — n=16 directions azimutales
    """
    try:
        import rvt.vis
        with rasterio.open(mnh_path) as src:
            arr = src.read(1).astype(float)
            res = src.res[0]
            nd  = src.nodata or -9999
        arr[arr == nd] = np.nan
        result = rvt.vis.sky_view_factor(
            dem=arr, resolution=res,
            compute_svf=True, compute_asvf=False, compute_opns=False)
        svf = result["svf"]
        print(f"OK  moy={np.nanmean(svf):.3f}  min={np.nanmin(svf):.3f}")
        return svf.astype(np.float32)
    except ImportError:
        print("⚠  rvt-py non disponible (pip install rvt-py)")
        return None
    except Exception as e:
        print(f"⚠  SVF : {e}")
        return None


def compute_hillshade_winter(mnt_data: np.ndarray,
                             cellsize: float = 0.5,
                             lat_deg: float = 45.57) -> np.ndarray:
    """
    Ombrage solaire au solstice d'hiver à midi (21 décembre, 12h solaire).

    Paramètres solaires (Chambéry lat=45.57°N) :
        Déclinaison :    δ = -23.45°
        Élévation midi : α = 90° − 45.57° − 23.45° = 20.98°
        Azimuth :        180° (plein sud, hémisphère nord)

    Formule hillshade standard :
        H = cos(z)·cos(s) + sin(z)·sin(s)·cos(θ_soleil − θ_aspect)
        z = angle zénithal solaire, s = slope, θ = azimuth

    Feature la plus différenciante du projet :
    un terrain orienté sud peut être entièrement à l'ombre en hiver
```

```python
    à cause du relief ou d'un immeuble voisin. Invisible sans LiDAR.
    Valeur 1 = plein soleil, 0 = ombre totale.
    """
    decl    = -23.45
    elev_rad = np.deg2rad(90.0 - lat_deg + decl)
    zen_rad  = np.pi / 2 - elev_rad
    az_rad   = np.pi  # 180° = plein sud

    dy, dx = np.gradient(mnt_data.astype(float), cellsize)
    slope_rad  = np.arctan(np.sqrt(dx**2 + dy**2))
    aspect_rad = np.arctan2(-dy, dx)

    hs = (np.cos(zen_rad) * np.cos(slope_rad)
          + np.sin(zen_rad) * np.sin(slope_rad) * np.cos(aspect_rad - az_rad))
    hs = np.clip(hs, 0, 1).astype(np.float32)
    hs[np.isnan(mnt_data)] = np.nan
    return hs


def compute_aspect_south(mnt_data: np.ndarray,
                         cellsize: float = 0.5) -> np.ndarray:
    """
    Ratio de pixels orientés sud (135°-225° depuis le nord).
    Agrégation zonale → proportion surface orientée sud par parcelle.
    Impact : performance RE2020, valorisation commerciale.
    """
    dy, dx = np.gradient(mnt_data.astype(float), cellsize)
    aspect_deg = np.degrees(np.arctan2(-dy, dx)) % 360
    is_south = ((aspect_deg >= 135) & (aspect_deg <= 225)).astype(np.float32)
    is_south[np.isnan(mnt_data)] = np.nan
    return is_south


def compute_profile_curvature(mnt_data: np.ndarray,
                              cellsize: float = 0.5) -> np.ndarray:
    """
    Courbure de profil (Laplacien normalisé).
    Positif = convexe → eau s'écoule vers l'extérieur → terrain stable.
    Négatif = concave → accumulation eaux → risque humidité fondations.
    Ref : Zevenbergen & Thorne (1987). Earth Surface Processes 12(1):47-56.
    """
    z = mnt_data.astype(float)
    d2z_dx2 = np.gradient(np.gradient(z, cellsize, axis=1), cellsize, axis=1)
    d2z_dy2 = np.gradient(np.gradient(z, cellsize, axis=0), cellsize, axis=0)
    curv = (d2z_dx2 + d2z_dy2).astype(np.float32)
    curv = uniform_filter(curv, size=5)  # Lissage bruit LiDAR
    curv[np.isnan(mnt_data)] = np.nan
    return curv


def save_tif(array: np.ndarray, ref_path: Path, out_path: Path) -> None:
    """Sauvegarde un array numpy en GeoTIFF avec CRS/transform de référence."""
    with rasterio.open(ref_path) as ref:
        meta = ref.meta.copy()
    meta.update({"count": 1, "dtype": "float32", "compress": "lzw", "nodata": -9999.0})
    arr = array.astype("float32")
    arr[np.isnan(arr)] = -9999.0
    with rasterio.open(out_path, "w", **meta) as dst:
        dst.write(arr, 1)


# ==============================================================================
# SECTION 4 — EXTRACTION ZONAL STATS
# ==============================================================================

def zonal(raster_path: Path, parcelles: gpd.GeoDataFrame,
          prefix: str, stats: list | None = None) -> dict:
    """Extraction de statistiques zonales par parcelle."""
    stats = stats or ["mean", "max", "std"]
    geoms = [g.__geo_interface__ for g in parcelles.geometry]
    try:
        res = zonal_stats(geoms, str(raster_path), stats=stats, nodata=-9999.0)
        return {f"{prefix}_{s}": [r.get(s) for r in res] for s in stats}
    except Exception as e:
```

```python
        print(f"    ⚠  zonal {prefix} : {e}")
        return {}


def zonal_percentile(raster_path: Path, parcelles: gpd.GeoDataFrame,
                     prefix: str, percentiles: list) -> dict:
    """
    Calcule des percentiles spécifiques par parcelle.
    Utilisé pour slope_p50 et slope_p90 — robustesse aux artefacts LiDAR.
    V3 : P50 et P90 plus importants que mean/max en milieu urbain dense.
    """
    stat_names = [f"percentile_{p}" for p in percentiles]
    geoms = [g.__geo_interface__ for g in parcelles.geometry]
    try:
        res = zonal_stats(geoms, str(raster_path),
                          stats=stat_names + ["std"], nodata=-9999.0)
        out = {}
        for p in percentiles:
            out[f"{prefix}_p{p}"] = [r.get(f"percentile_{p}") for r in res]
        out[f"{prefix}_std"] = [r.get("std") for r in res]
        return out
    except Exception as e:
        print(f"    ⚠  zonal percentiles {prefix} : {e}")
        return {}


# ==============================================================================
# SECTION 5 — FILTRAGE ET NETTOYAGE
# ==============================================================================

def filter_parcelles(df: pd.DataFrame,
                     parcelles: gpd.GeoDataFrame) -> pd.DataFrame:
    """
    Filtre les parcelles problématiques identifiées en V1/V2.
    Seuil NAN assoupli à 40% en V3 (vs 50% V2) pour récupérer plus de parcelles.
    """
    lidar_cols = [c for c in ALL_FEATURES if c in df.columns]
    df["nan_ratio"]  = df[lidar_cols].isna().mean(axis=1) if lidar_cols else 0.0
    df["surface_m2"] = parcelles.geometry.area.values

    df["is_valid"] = (
        (df["nan_ratio"]  <= SEUIL_NAN) &
        (df["surface_m2"] <= SEUIL_SURFACE) &
        (df["surface_m2"] >= SEUIL_SURF_MIN)
    )

    n_tot   = len(df)
    n_valid = df["is_valid"].sum()
    n_nan   = (df["nan_ratio"] > SEUIL_NAN).sum()
    n_big   = (df["surface_m2"] > SEUIL_SURFACE).sum()
    n_small = (df["surface_m2"] < SEUIL_SURF_MIN).sum()

    print(f"    Total               : {n_tot:,}")
    print(f"    ✓ Valides           : {n_valid:,} ({n_valid/n_tot*100:.1f}%)")
    print(f"    ✗ NaN > {SEUIL_NAN*100:.0f}%        : {n_nan:,} ({n_nan/n_tot*100:.1f}%)")
    print(f"    ✗ > {SEUIL_SURFACE:.0f}m²       : {n_big:,}")
    print(f"    ✗ < {SEUIL_SURF_MIN:.0f}m²          : {n_small:,}")

    return df


# ==============================================================================
# SECTION 6 — SCORE CPI V3 DÉTERMINISTE
# ==============================================================================

def calibrate_tau(group_scores_df: pd.DataFrame,
                  target_std: float = 18.0) -> float:
    """
    Calibration automatique de τ (température softmin).

    Principe : on cherche τ tel que l'écart-type du score final soit ~18 pts,
    ce qui donne une distribution [0,100] raisonnablement étalée.
    - τ trop faible → scores extrêmes, distribution bimodale
    - τ trop élevé  → scores trop lisses, tout proche de la moyenne
```

```python
    Méthode : recherche binaire sur τ ∈ [0.1, 50].

    V3 améliore la V2 où τ=15 donnait 94.8% en "Moyen" — trop lisse.
    """
    groups = group_scores_df.values  # shape (n, 4)

    def score_std(tau):
        v = groups.astype(float)
        v_s = v - np.max(v, axis=1, keepdims=True)
        exp_neg = np.exp(-v_s / tau)
        w = exp_neg / exp_neg.sum(axis=1, keepdims=True)
        sm = (w * v).sum(axis=1)
        combined = 0.70 * v.mean(axis=1) * 0 + 0.30 * sm  # simplified
        combined = 0.70 * groups.mean(axis=1) + 0.30 * sm
        return combined.std()

    lo, hi = 0.5, 100.0
    for _ in range(50):
        mid = (lo + hi) / 2
        s = score_std(mid)
        if s < target_std:
            hi = mid
        else:
            lo = mid

    tau_opt = (lo + hi) / 2
    print(f"    τ calibré : {tau_opt:.2f}  (std visée={target_std}, obtenue={score_std(tau_opt):.2f})")
    return tau_opt


def softmin(values: np.ndarray, tau: float) -> np.ndarray:
    """
    Softmin différentiable avec paramètre τ calibré.
    Ref : recommandation IA mathématiques (Mars 2026).
    τ→0 : converge vers min(x). τ→∞ : converge vers mean(x).
    Stabilisation numérique par décalage max.
    """
    v = np.array(values, dtype=float)
    if v.ndim == 1:
        v = v.reshape(1, -1)
    v_s   = v - np.max(v, axis=1, keepdims=True)
    exp_n = np.exp(-v_s / max(tau, 1e-9))
    w     = exp_n / exp_n.sum(axis=1, keepdims=True)
    return (w * v).sum(axis=1)


def normalize(series: pd.Series, invert: bool = False,
              pct_low: float = 1.0, pct_high: float = 99.0) -> pd.Series:
    """
    Normalisation [0,1] robuste aux outliers.
    V3 : utilise les percentiles P1/P99 au lieu du min/max strict
    pour éviter qu'un pixel aberrant LiDAR comprime toute la distribution.
    """
    valid = series.dropna()
    mn = np.percentile(valid, pct_low)  if len(valid) > 0 else 0
    mx = np.percentile(valid, pct_high) if len(valid) > 0 else 1
    if mx == mn:
        return pd.Series(0.5, index=series.index)
    n = (series.fillna(mn).clip(mn, mx) - mn) / (mx - mn)
    return 1 - n if invert else n


def compute_cpi_v3(df: pd.DataFrame, tau: float) -> pd.DataFrame:
    """
    Score CPI V3 — architecture deux niveaux avec τ calibré.

    Niveau 1 — Gates d'élimination
        has_thalweg_mean > 0.20  →  CPI ≤ 30
        slope_p90        > 25°   →  CPI ≤ 25

    Niveau 2 — Score continu (normalisation P1/P99 robuste)
        PENTE (35%)          : 0.40×f(p50) + 0.35×f(p90) + 0.25×f(std)
        HYDROLOGIE (30%)     : f(twi_mean)  (thalweg dans gate)
```

```
    MORPHOLOGIE (20%)   : 0.55×f(curv) + 0.45×f(tri)
    ENSOLEILLEMENT (15%): 0.40×f(aspect) + 0.35×f(hillshade) + 0.25×f(svf)

Formule finale :
    CPI = gate × [0.70 × moy_pond + 0.30 × softmin(groupes, τ_calibré)]
"""
def get(col, default=0.5):
    if col in df.columns:
        return df[col].fillna(
            df[col].median() if df[col].notna().any() else default)
    return pd.Series(default, index=df.index)

# ── Scores par groupe [0,1]
sp = (0.40 * normalize(get("slope_p50"), invert=True)
    + 0.35 * normalize(get("slope_p90"), invert=True)
    + 0.25 * normalize(get("slope_std"), invert=True))

sh = normalize(get("twi_mean"), invert=True)
if "has_thalweg_mean" in df.columns:
    thalw = df["has_thalweg_mean"].fillna(0).clip(0, 1)
    sh = sh * (1 - 0.4 * thalw)

sm = (0.55 * normalize(get("profile_curvature_mean"), invert=False)
    + 0.45 * normalize(get("tri_mean"), invert=True))

# Ensoleillement V3 : 3 features (vs 2 en V2)
has_svf = "svf_mean" in df.columns
if has_svf:
    se = (0.40 * normalize(get("aspect_south_ratio_mean"))
        + 0.35 * normalize(get("hillshade_winter_mean"))
        + 0.25 * normalize(get("svf_mean")))
else:
    se = (0.57 * normalize(get("aspect_south_ratio_mean"))
        + 0.43 * normalize(get("hillshade_winter_mean")))

df["score_pente"]   = (sp * 100).round(1)
df["score_hydro"]   = (sh * 100).round(1)
df["score_morpho"]  = (sm * 100).round(1)
df["score_soleil"]  = (se * 100).round(1)

# ── Moyenne pondérée
w = GROUP_WEIGHTS
df["score_continu"] = (
    w["PENTE"] * df["score_pente"] + w["HYDROLOGIE"] * df["score_hydro"]
  + w["MORPHOLOGIE"] * df["score_morpho"] + w["ENSOLEILLEMENT"] * df["score_soleil"]
).round(1)

# ── Softmin avec τ calibré
gscores = df[["score_pente","score_hydro","score_morpho","score_soleil"]].values
df["score_softmin"] = softmin(gscores, tau).round(1)

# ── Score brut
df["cpi_brut"] = (0.70 * df["score_continu"] + 0.30 * df["score_softmin"]).round(1)

# ── Niveau 1 : Gates
df["gate_factor"] = 1.0
df["gate_reason"] = ""

if "has_thalweg_mean" in df.columns:
    m = df["has_thalweg_mean"].fillna(0) > 0.20
    df.loc[m, "gate_factor"] = np.minimum(
        df.loc[m, "gate_factor"],
        30.0 / df.loc[m, "cpi_brut"].clip(lower=1))
    df.loc[m, "gate_reason"] = "talweg_central"

if "slope_p90" in df.columns:
    m = df["slope_p90"].fillna(0) > 25.0
    df.loc[m, "gate_factor"] = np.minimum(
        df.loc[m, "gate_factor"],
        25.0 / df.loc[m, "cpi_brut"].clip(lower=1))
    df.loc[m & (df["gate_reason"] == ""), "gate_reason"] = "pente_extreme"
    df.loc[m & (df["gate_reason"] != ""), "gate_reason"] = "talweg+pente"

df["CPI_v3"] = (df["cpi_brut"] * df["gate_factor"]).clip(0, 100).round(1)
```

```python
    def interpret(s):
        if s < 25: return "Éliminatoire"
        if s < 45: return "Faible"
        if s < 65: return "Moyen"
        if s < 85: return "Bon"
        return "Excellent"

    df["CPI_v3_label"] = df["CPI_v3"].apply(interpret)

    n_valid = df["is_valid"].sum() if "is_valid" in df.columns else len(df)
    valid_cpi = df.loc[df.get("is_valid", pd.Series(True, index=df.index)), "CPI_v3"]
    print(f"    CPI_v3 (parcelles valides) — "
          f"moy={valid_cpi.mean():.1f}  std={valid_cpi.std():.1f}  "
          f"min={valid_cpi.min():.1f}  max={valid_cpi.max():.1f}")
    print(f"    Gates : {(df['gate_factor'] < 1).sum()} parcelles plafonnées")
    print(f"    Distribution :")
    for lbl in ["Éliminatoire","Faible","Moyen","Bon","Excellent"]:
        n = (df["CPI_v3_label"] == lbl).sum()
        print(f"      {lbl:<15}: {n:>5} ({n/len(df)*100:.1f}%)")

    return df


# ==============================================================================
# SECTION 7 — LABELS SNORKEL V3
# ==============================================================================

def prepare_dvf(dvf_path: Path) -> pd.DataFrame | None:
    """
    Prépare les données DVF pour les heuristiques de labelling.
    Calcule le prix/m² par parcelle et les percentiles P20/P80.

    Colonnes DVF attendues (format data.gouv.fr) :
        id_parcelle      : identifiant cadastral
        valeur_fonciere  : prix de vente (€)
        surface_terrain  : surface du terrain (m²)
        nature_mutation  : type de transaction (Vente, ...)

    Ref : Demandes de valeur foncière, data.gouv.fr, Étalab 2.0.
    """
    if not dvf_path.exists():
        return None
    try:
        df = pd.read_csv(dvf_path, low_memory=False)
        print(f"    DVF chargé : {len(df)} lignes")

        # Normalisation colonnes — on prend la PREMIÈRE colonne qui matche
        rename = {}
        already_renamed = set()
        for c in df.columns:
            cl = c.lower().replace(" ", "_")
            if "valeur" in cl and "fonci" in cl and "valeur_fonciere" not in already_renamed:
                rename[c] = "valeur_fonciere"; already_renamed.add("valeur_fonciere")
            elif "surface" in cl and "terrain" in cl and "surface_terrain" not in already_renamed:
                rename[c] = "surface_terrain"; already_renamed.add("surface_terrain")
            elif ("id_parcelle" in cl or "code_parcelle" in cl) and "id_parcelle" not in already_renamed:
                rename[c] = "id_parcelle"; already_renamed.add("id_parcelle")
            elif "nature" in cl and "mutation" in cl and "nature_mutation" not in already_renamed:
                rename[c] = "nature_mutation"; already_renamed.add("nature_mutation")
        df = df.rename(columns=rename)

        # Filtrer sur les ventes de terrains uniquement
        if "nature_mutation" in df.columns:
            df = df[df["nature_mutation"].str.contains("Vente", na=False)]

        if "valeur_fonciere" not in df.columns:
            print("    ⚠  DVF : colonne valeur_fonciere introuvable")
            return None

        df["valeur_fonciere"] = pd.to_numeric(df["valeur_fonciere"], errors="coerce")
        df["surface_terrain"] = pd.to_numeric(
            df.get("surface_terrain", pd.Series()), errors="coerce")
        df = df.dropna(subset=["valeur_fonciere"])
```

```python
        df = df[df["valeur_fonciere"] > 0]

        if "surface_terrain" in df.columns and df["surface_terrain"].notna().any():
            df = df[df["surface_terrain"] > 0]
            df["prix_m2"] = df["valeur_fonciere"] / df["surface_terrain"]
            df = df[df["prix_m2"].between(10, 10000)]  # prix réalistes

        p20 = df["prix_m2"].quantile(0.20) if "prix_m2" in df.columns else None
        p80 = df["prix_m2"].quantile(0.80) if "prix_m2" in df.columns else None
        p20_str = f"{p20:.0f}" if p20 is not None else "N/A"
        p80_str = f"{p80:.0f}" if p80 is not None else "N/A"
        med = df.get('prix_m2', pd.Series()).median()
        med_str = f"{med:.0f}" if pd.notna(med) else "N/A"
        print(f"    DVF propre : {len(df)} ventes | "
              f"prix médian={med_str}€/m² | P20={p20_str}€ P80={p80_str}€")
        return df

    except Exception as e:
        print(f"    ⚠  DVF erreur : {e}")
        return None


def create_snorkel_labels_v3(df: pd.DataFrame,
                             dvf: pd.DataFrame | None = None) -> pd.Series:
    """
    Labels Snorkel V3 — 4 heuristiques avec vote pondéré.

    Ref : Ratner, A.J. et al. (2017). Snorkel: Rapid Training Data Creation
          with Weak Supervision. VLDB Journal, 26:793-817.

    HEURISTIQUES
    ────────────────────────────────────────────────────────────────────────
    H1 LiDAR_POS — Constructible probable (confiance MOYENNE)
        slope_p50 < 8° ET surface > 200m² ET svf > 0.35
        Ref : seuil G1 géotechnique France (NF P 94-500)

    H2 LiDAR_NEG — Non constructible (confiance HAUTE)
        slope_p90 > 15° OU surface < 100m²
        Ref : consensus géotechniciens + pratique promoteurs FR

    H3 DVF_POS — Terrain valorisé par le marché (confiance HAUTE)
        prix_m² > P80 commune → signal que des pros ont jugé constructible
        Indépendant des features LiDAR → résout redondance V2

    H4 DVF_NEG — Terrain peu valorisé (confiance MOYENNE)
        prix_m² < P20 commune → potentiel jugé faible
        Attention : peut inclure terrains agricoles/industriels

    FUSION (vote pondéré) :
        pos_score = 0.5×H1 + 0.5×H3  (H1 moins fiable car redondant LiDAR)
        neg_score = 0.7×H2 + 0.3×H4
        Si pos_score > neg_score ET pos_score > 0.4 → label 1
        Si neg_score > pos_score ET neg_score > 0.4 → label 0
        Sinon → ABSTAIN (-1)
    """
    n = len(df)
    h1 = pd.Series(0.0, index=df.index)
    h2 = pd.Series(0.0, index=df.index)
    h3 = pd.Series(0.0, index=df.index)
    h4 = pd.Series(0.0, index=df.index)

    slope_p50 = df.get("slope_p50", pd.Series(5.0, index=df.index)).fillna(5.0)
    slope_p90 = df.get("slope_p90", pd.Series(8.0, index=df.index)).fillna(8.0)
    surface   = df.get("surface_m2", pd.Series(300.0, index=df.index)).fillna(300.0)
    svf_      = df.get("svf_mean",   pd.Series(0.7,   index=df.index)).fillna(0.7)

    # H1 : signal LiDAR constructible
    h1[(slope_p50 < 8.0) & (surface > 200.0) & (svf_ > 0.35)] = 1.0

    # H2 : signal LiDAR contrainte forte
    h2[(slope_p90 > 15.0) | (surface < 100.0)] = 1.0

    # H3 & H4 : DVF
    dvf_active = False
```

```python
    if dvf is not None and "prix_m2" in dvf.columns and len(dvf) > 0:
        p80 = dvf["prix_m2"].quantile(0.80)
        p20 = dvf["prix_m2"].quantile(0.20)

        # Jointure par id_parcelle si disponible
        if "id_parcelle" in dvf.columns and "id" in df.columns:
            # S'assurer que id_parcelle est bien 1D (pas de doublons de colonnes)
            dvf_clean = dvf.copy()
            if isinstance(dvf_clean["id_parcelle"], pd.DataFrame):
                dvf_clean["id_parcelle"] = dvf_clean["id_parcelle"].iloc[:, 0]
            dvf_clean["id_parcelle"] = dvf_clean["id_parcelle"].astype(str).str.strip()
            prix_parcel = dvf_clean.groupby("id_parcelle")["prix_m2"].median()
            df_prix = df["id"].astype(str).str.strip().map(prix_parcel)
            h3[df_prix > p80] = 1.0
            h4[df_prix < p20] = 1.0
            dvf_active = True
            print(f"    DVF actif : H3={h3.sum():.0f} positifs | H4={h4.sum():.0f} négatifs")
        else:
            print("    ⚠  DVF : pas de jointure id_parcelle — H3/H4 ignorées")

    # Vote pondéré
    pos_score = 0.5 * h1 + (0.5 * h3 if dvf_active else 0.0)
    neg_score = 0.7 * h2 + (0.3 * h4 if dvf_active else 0.0)

    threshold = 0.35  # légèrement abaissé si DVF absent
    if not dvf_active:
        threshold = 0.45  # plus strict sans DVF

    labels = pd.Series(-1, index=df.index)
    labels[(pos_score > neg_score) & (pos_score >= threshold)] = 1
    labels[(neg_score >= pos_score) & (neg_score >= threshold)] = 0

    n_pos = (labels == 1).sum()
    n_neg = (labels == 0).sum()
    n_abs = (labels == -1).sum()
    print(f"    Labels finaux : {n_pos} pos / {n_neg} neg / {n_abs} abstain")
    print(f"    Ratio pos/neg : {n_pos/(n_neg+1e-9):.2f}")
    return labels


# ==============================================================================
# SECTION 8 — SPATIAL CV + ANALYSE BLOCS
# ==============================================================================

def create_spatial_blocks(parcelles: gpd.GeoDataFrame,
                          grid_size: int = GRID_SIZE_M) -> pd.Series:
    """
    Blocs spatiaux pour GroupKFold — grille régulière {grid_size}m.
    V3 : 300m (vs 200m V2) pour meilleure indépendance spatiale.

    block_id = "{int(x/grid_size)}_{int(y/grid_size)}"

    Ref : Valavi et al. (2019). blockCV. Methods Ecol Evol 10(2):225-232.
    """
    c = parcelles.geometry.centroid
    bx = (c.x / grid_size).astype(int)
    by = (c.y / grid_size).astype(int)
    return (bx.astype(str) + "_" + by.astype(str)).rename("block_id")


def analyze_blocks(df: pd.DataFrame, labels: pd.Series) -> dict:
    """
    Analyse de l'équilibre des classes par bloc spatial.
    Détecte les blocs monoclasses qui biaiseraient la CV.

    V3 : recommandation IA math (Mars 2026) — vérifier avant la CV.
    """
    mask = labels != -1
    valid = df[mask].copy()
    valid["label"] = labels[mask].values

    if "block_id" not in valid.columns:
        return {}
```

```python
    stats = valid.groupby("block_id")["label"].agg(["sum", "count"])
    stats["ratio"] = stats["sum"] / stats["count"]

    pure_pos = (stats["ratio"] > 0.90).sum()
    pure_neg = (stats["ratio"] < 0.10).sum()
    mixed    = ((stats["ratio"] >= 0.10) & (stats["ratio"] <= 0.90)).sum()

    print(f"    Analyse blocs {GRID_SIZE_M}m :")
    print(f"      Blocs mixtes (10-90%)    : {mixed}")
    print(f"      Blocs purs positifs >90% : {pure_pos}")
    print(f"      Blocs purs négatifs <10% : {pure_neg}")
    if pure_pos + pure_neg > 0.3 * len(stats):
        print(f"    ⚠  Beaucoup de blocs monoclasses — "
              f"essayer {GRID_SIZE_M + 100}m ou plus pour améliorer")

    return {"mixed": int(mixed), "pure_pos": int(pure_pos),
            "pure_neg": int(pure_neg), "total": int(len(stats))}


# ==============================================================================
# SECTION 9 — COMPARAISON DES MODÈLES ML V3
# ==============================================================================

def precision_at_k(y_true: np.ndarray, y_score: np.ndarray, k: int = 20) -> float:
    k_a = min(k, len(y_true))
    return float(y_true[np.argsort(y_score)[-k_a:]].mean()) if k_a > 0 else 0.0


def ndcg_at_k(y_true: np.ndarray, y_score: np.ndarray, k: int = 20) -> float:
    k_a = min(k, len(y_true))
    if k_a == 0:
        return 0.0
    order  = np.argsort(y_score)[::-1][:k_a]
    gains  = y_true[order].astype(float)
    disc   = np.log2(np.arange(2, k_a + 2))
    dcg    = np.sum(gains / disc)
    ideal  = np.sort(y_true.astype(float))[::-1][:k_a]
    idcg   = np.sum(ideal / disc)
    return float(dcg / idcg) if idcg > 0 else 0.0


def compare_models_v3(df: pd.DataFrame,
                      labels: pd.Series,
                      parcelles: gpd.GeoDataFrame) -> dict:
    """
    Comparaison rigoureuse des modèles avec GroupKFold spatial + GridSearch XGBoost.

    Améliorations V3 :
        - qid=block_id dans XGBRanker (groupes locaux, pas global)
        - GridSearch sur XGBRanker (n_estimators, max_depth, learning_rate)
        - Matrice de confusion par fold pour RF (diagnostic NDCG=1.000)
        - Spearman correlation scores vs CPI_v3 (validation externe)

    V3 : XGBoost avec groupes par bloc spatial donne un ranking par quartier,
    ce qui correspond à l'usage réel (promoteur compare des parcelles dans
    un même secteur géographique).
    """
    try:
        from sklearn.linear_model import LogisticRegression
        from sklearn.ensemble import RandomForestClassifier
        from sklearn.preprocessing import StandardScaler
        from sklearn.model_selection import GroupKFold, ParameterGrid
        from sklearn.metrics import roc_auc_score, confusion_matrix
        import xgboost as xgb
    except ImportError as e:
        print(f"  ✗ Import manquant : {e}")
        return {}

    mask      = (labels != -1) & df.get("is_valid", pd.Series(True, index=df.index))
    feat_cols = [c for c in ALL_FEATURES if c in df.columns]

    if not feat_cols or mask.sum() < 100:
        print(f"  ✗ Données insuffisantes ({mask.sum()} exemples labellisés)")
        return {}
```

```python
X_full = df[feat_cols].copy()
for col in feat_cols:
    X_full[col] = X_full[col].fillna(X_full[col].median())

X       = X_full[mask].values
y       = labels[mask].values
blocks  = df.loc[mask, "block_id"].values if "block_id" in df.columns \
          else np.zeros(mask.sum())
cpi_ref = df.loc[mask, "CPI_v3"].values if "CPI_v3" in df.columns else None

print(f"    Dataset : {len(X):,} parcelles ({y.sum()} pos / {(y==0).sum()} neg)")
print(f"    Features : {feat_cols}")
print(f"    Blocs : {len(np.unique(blocks))} blocs de {GRID_SIZE_M}m")

gkf = GroupKFold(n_splits=min(N_FOLDS, len(np.unique(blocks))))

# ── GridSearch XGBRanker
print("\n    ⚙  GridSearch XGBRanker ...", flush=True)
param_grid = {
    "n_estimators":  [100, 200, 300],
    "max_depth":     [3, 4, 5],
    "learning_rate": [0.03, 0.05, 0.1],
}
best_ndcg, best_params = 0, {}

for params in ParameterGrid(param_grid):
    ndcgs = []
    model = xgb.XGBRanker(
        objective="rank:ndcg", subsample=0.8,
        colsample_bytree=0.8, random_state=42, n_jobs=-1,
        **params)
    for tr_idx, val_idx in gkf.split(X, y, groups=blocks):
        Xtr, Xval = X[tr_idx], X[val_idx]
        ytr, yval = y[tr_idx], y[val_idx]
        # Groupes par bloc pour LambdaMART
        blocks_tr  = blocks[tr_idx]
        _, qid_tr  = np.unique(blocks_tr, return_inverse=True)
        # Trier par qid pour XGBoost
        sort_idx   = np.argsort(qid_tr)
        Xtr_sorted = Xtr[sort_idx]
        ytr_sorted = ytr[sort_idx]
        qid_sorted = qid_tr[sort_idx]
        # Compter éléments par groupe
        _, counts  = np.unique(qid_sorted, return_counts=True)
        model.fit(Xtr_sorted, ytr_sorted, group=counts)
        scores = model.predict(Xval)
        ndcgs.append(ndcg_at_k(yval, scores, k=min(20, len(yval))))
    mean_ndcg = np.mean(ndcgs)
    if mean_ndcg > best_ndcg:
        best_ndcg   = mean_ndcg
        best_params = params

print(f"    XGBRanker best params : {best_params}  NDCG@20={best_ndcg:.4f}")

# ── Comparaison finale avec les meilleurs params
models = {
    "Logistic Regression": LogisticRegression(max_iter=2000, random_state=42),
    "Random Forest":       RandomForestClassifier(
                                n_estimators=300, max_depth=6,
                                random_state=42, n_jobs=-1),
    "XGBoost rank:ndcg":   xgb.XGBRanker(
                                objective="rank:ndcg", subsample=0.8,
                                colsample_bytree=0.8, random_state=42, n_jobs=-1,
                                **best_params),
}

results  = {}
rf_cms   = []   # Matrices confusion RF par fold

for name, model in models.items():
    aucs, p20s, ndcgs, spearmans = [], [], [], []

    for tr_idx, val_idx in gkf.split(X, y, groups=blocks):
```

```
        Xtr, Xval = X[tr_idx], X[val_idx]
        ytr, yval = y[tr_idx], y[val_idx]

        if name == "Logistic Regression":
            sc   = StandardScaler()
            Xtr  = sc.fit_transform(Xtr)
            Xval = sc.transform(Xval)
            model.fit(Xtr, ytr)
            scores = model.predict_proba(Xval)[:, 1]

        elif name == "Random Forest":
            model.fit(Xtr, ytr)
            scores = model.predict_proba(Xval)[:, 1]
            # Matrice de confusion — diagnostic NDCG=1.000 V2
            preds = model.predict(Xval)
            rf_cms.append(confusion_matrix(yval, preds, labels=[0, 1]))

        else:  # XGBoost rank:ndcg avec groupes bloc
            blocks_tr = blocks[tr_idx]
            _, qid_tr = np.unique(blocks_tr, return_inverse=True)
            sort_idx  = np.argsort(qid_tr)
            Xtr_s     = Xtr[sort_idx]
            ytr_s     = ytr[sort_idx]
            _, cnts   = np.unique(qid_tr[sort_idx], return_counts=True)
            model.fit(Xtr_s, ytr_s, group=cnts)
            scores = model.predict(Xval)

        if len(np.unique(yval)) > 1:
            try:
                aucs.append(roc_auc_score(yval, scores))
            except Exception:
                pass

        p20s.append(precision_at_k(yval, scores, k=min(20, len(yval))))
        ndcgs.append(ndcg_at_k(yval, scores, k=min(20, len(yval))))

        # Corrélation Spearman vs CPI_v3 (validation externe indépendante)
        if cpi_ref is not None:
            r, _ = spearmanr(scores, cpi_ref[val_idx])
            spearmans.append(r)

    results[name] = {
        "AUC":          f"{np.mean(aucs):.3f} ± {np.std(aucs):.3f}" if aucs else "N/A",
        "Precision@20": f"{np.mean(p20s):.3f} ± {np.std(p20s):.3f}",
        "NDCG@20":      f"{np.mean(ndcgs):.3f} ± {np.std(ndcgs):.3f}",
        "Spearman_CPI": f"{np.mean(spearmans):.3f}" if spearmans else "N/A",
        "_ndcg_mean":   np.mean(ndcgs),
        "_auc_mean":    np.mean(aucs) if aucs else 0,
    }

# Matrice de confusion RF agrégée
if rf_cms:
    cm_avg = np.array(rf_cms).mean(axis=0).astype(int)
    tn, fp, fn, tp = cm_avg.ravel()
    print(f"\n    Matrice confusion RF (moy. folds) :")
    print(f"      Vrais Négatifs  : {tn:>5}  |  Faux Positifs : {fp:>5}")
    print(f"      Faux Négatifs   : {fn:>5}  |  Vrais Positifs: {tp:>5}")
    print(f"      Précision : {tp/(tp+fp+1e-9):.3f}  Rappel : {tp/(tp+fn+1e-9):.3f}")
    results["RF_confusion_matrix"] = cm_avg.tolist()

best = max(results,
           key=lambda k: results[k].get("_ndcg_mean", 0)
           if k != "RF_confusion_matrix" else -1)

print(f"\n    {'Modèle':<25} {'AUC':>14} {'P@20':>16} {'NDCG@20':>16} {'Spearman':>12}")
print(f"    {'-'*88}")
for name, r in results.items():
    if name == "RF_confusion_matrix":
        continue
    marker = "  ← meilleur NDCG" if name == best else ""
    print(f"    {name:<25} {r['AUC']:>14} {r['Precision@20']:>16} "
          f"{r['NDCG@20']:>16} {r['Spearman_CPI']:>12}{marker}")

print(f"\n    → Modèle retenu     : {best}")
```

```python
    print(f"    → Critère principal : NDCG@20 (qualité ranking — usage métier)")
    if spearmans:
        print(f"    → Validation externe: Spearman vs CPI_v3 déterministe")

    return results


# ==============================================================================
# SECTION 10 — XGBOOST LAMBDAMART FINAL + SHAP V3
# ==============================================================================

def train_and_explain_v3(df: pd.DataFrame,
                         labels: pd.Series,
                         best_params: dict | None = None) -> None:
    """
    Entraînement XGBoost LambdaMART final + SHAP complet.

    Améliorations V3 :
        - Hyperparamètres issus du GridSearch
        - qid=block_id (groupes par contexte local)
        - SHAP par groupe thématique (PENTE, HYDRO, MORPHO, SOLEIL)
        - Analyse importance relative des groupes
        - Détection des parcelles les plus "mal expliquées" (écart CPI_v3 vs ML)

    Ref SHAP : Lundberg, S.M., Lee, S.I. (2017).
               A Unified Approach to Interpreting Model Predictions. NeurIPS 30.
    Ref LambdaMART : Burges, C. et al. (2006). Learning to rank. ICML.
    """
    try:
        import xgboost as xgb
        import shap
    except ImportError:
        print("  ✗ xgboost ou shap manquant")
        return

    mask      = (labels != -1) & df.get("is_valid", pd.Series(True, index=df.index))
    feat_cols = [c for c in ALL_FEATURES if c in df.columns]

    if not feat_cols or mask.sum() < 50:
        print("  ✗ Données insuffisantes"); return

    X_full = df[feat_cols].copy()
    for col in feat_cols:
        X_full[col] = X_full[col].fillna(X_full[col].median())

    X_lab  = X_full[mask]
    y_lab  = labels[mask].values
    bl_lab = df.loc[mask, "block_id"].values if "block_id" in df.columns \
             else np.zeros(len(y_lab))

    # Groupes pour LambdaMART
    _, qid_tr  = np.unique(bl_lab, return_inverse=True)
    sort_idx   = np.argsort(qid_tr)
    X_sorted   = X_lab.values[sort_idx]
    y_sorted   = y_lab[sort_idx]
    _, cnts    = np.unique(qid_tr[sort_idx], return_counts=True)

    params = best_params or {"n_estimators": 300, "max_depth": 4, "learning_rate": 0.05}

    model = xgb.XGBRanker(
        objective="rank:ndcg",
        subsample=0.8, colsample_bytree=0.8,
        random_state=42, n_jobs=-1, **params)
    model.fit(X_sorted, y_sorted, group=cnts)

    # Score ML sur parcelles valides
    valid_mask = df.get("is_valid", pd.Series(True, index=df.index))
    scores     = model.predict(X_full[valid_mask].values)
    s_min, s_max = scores.min(), scores.max()
    scores_norm  = ((scores - s_min) / (s_max - s_min + 1e-9) * 100).round(1) \
                   if s_max > s_min else np.full_like(scores, 50.0)
    df["CPI_ML_v3"] = np.nan
    df.loc[valid_mask, "CPI_ML_v3"] = scores_norm
```

```python
    # ── SHAP
    explainer   = shap.TreeExplainer(model)
    shap_values = explainer.shap_values(X_lab.values)

    shap_imp = pd.DataFrame({
        "feature":     feat_cols,
        "mean_|SHAP|": np.abs(shap_values).mean(axis=0),
        "groupe":      [next((g for g, fs in FEATURE_GROUPS.items()
                              if f in fs), "?") for f in feat_cols],
    }).sort_values("mean_|SHAP|", ascending=False)

    # ── Importance par groupe (agrégé)
    group_imp = shap_imp.groupby("groupe")["mean_|SHAP|"].sum().sort_values(ascending=False)

    print(f"\n    ── SHAP Importance globale des features ──")
    print(f"    {'Feature':<35} {'Groupe':<15} {'|SHAP|':>10}  Barre")
    print(f"    {'-'*75}")
    max_s = shap_imp["mean_|SHAP|"].max()
    for _, row in shap_imp.iterrows():
        bar = "▇" * max(1, int(row["mean_|SHAP|"] / max(max_s, 1e-9) * 25))
        print(f"    {row['feature']:<35} {row['groupe']:<15} "
              f"{row['mean_|SHAP|']:>10.4f}  {bar}")

    print(f"\n    ── SHAP Importance par groupe thématique ──")
    for grp, val in group_imp.items():
        w_spec = GROUP_WEIGHTS.get(grp, 0)
        bar    = "▇" * max(1, int(val / max(group_imp.max(), 1e-9) * 20))
        print(f"    {grp:<20} SHAP={val:.4f}  Poids_spec={w_spec:.0%}  {bar}")

    # ── Parcelle meilleure
    best_idx = df["CPI_ML_v3"].dropna().idxmax()
    best_sc  = df.loc[best_idx, "CPI_ML_v3"]
    lab_list = df[mask].index.tolist()
    if best_idx in lab_list:
        i = lab_list.index(best_idx)
        bs = pd.Series(shap_values[i], index=feat_cols).sort_values()
        print(f"\n    ── SHAP décomposition meilleure parcelle (CPI_ML={best_sc:.1f}) ──")
        for feat, val in bs.items():
            grp  = next((g for g, fs in FEATURE_GROUPS.items() if feat in fs), "?")
            sign = "+" if val >= 0 else ""
            col  = "🟢" if val >= 0 else "🔴"
            print(f"    {col} {feat:<35} {sign}{val:>8.4f}  [{grp}]")

    # ── Corrélation vs CPI_v3
    if "CPI_v3" in df.columns:
        common = df[["CPI_v3", "CPI_ML_v3"]].dropna()
        if len(common) > 10:
            corr_pearson = common["CPI_v3"].corr(common["CPI_ML_v3"])
            corr_spear   = spearmanr(common["CPI_v3"], common["CPI_ML_v3"])[0]
            diff         = (common["CPI_ML_v3"] - common["CPI_v3"]).abs()
            print(f"\n    ── CPI_v3 (deterministe) vs CPI_ML_v3 (LambdaMART) ──")
            print(f"    Pearson  : {corr_pearson:.3f}")
            print(f"    Spearman : {corr_spear:.3f}  "
                  f"(rang > valeur pour un ranking)")
            print(f"    Écart moy: {diff.mean():.1f} pts  max={diff.max():.1f} pts")
            if corr_spear > 0.7:
                print("    → Bonne cohérence : ML confirme le score déterministe")
            else:
                print("    → Divergence : ML capture interactions non-linéaires")

    # Sauvegarder SHAP
    shap_imp.to_csv(DATA_DIR / "shap_importance_v3.csv", index=False)
    group_imp.to_csv(DATA_DIR / "shap_importance_groupes_v3.csv", header=True)
    print(f"\n    SHAP sauvegardé → {DATA_DIR}/shap_importance_v3.csv")


# ============================================================================
# SECTION 11 — EXPORT
# ============================================================================

def export_v3(df: pd.DataFrame, parcelles: gpd.GeoDataFrame,
              stats_report: dict) -> None:
    """
    Export trois fichiers + rapport JSON.
```

```
    1. features_parcelles_v3.csv — Dataset complet
    2. ml_dataset_v3.csv         — Dataset propre coéquipier ML
    3. rapport_stats_v3.json     — Rapport machine-readable pour monitoring
    """
    score_cols = ["CPI_v3", "CPI_v3_label", "CPI_ML_v3",
                  "score_pente", "score_hydro", "score_morpho", "score_soleil",
                  "score_continu", "score_softmin", "gate_factor", "gate_reason"]
    feat_cols  = [c for c in ALL_FEATURES if c in df.columns]
    meta_cols  = ["commune", "surface_m2", "nan_ratio", "is_valid",
                  "proxy_label", "block_id"]
    all_cols   = [c for c in meta_cols + feat_cols + score_cols if c in df.columns]

    df_export = df.copy()
    if "id" in parcelles.columns:
        df_export.insert(0, "id_parcelle", parcelles["id"].values)

    df_export[all_cols].to_csv(OUTPUT_CSV_V3, index=True)
    print(f"\n  ✓ Dataset complet : {OUTPUT_CSV_V3}")
    print(f"    {len(df_export):,} parcelles × {len(all_cols)} colonnes")

    # Dataset ML
    valid_labeled = (df["is_valid"] & (df["proxy_label"] != -1)) \
                    if "proxy_label" in df.columns and "is_valid" in df.columns \
                    else pd.Series(False, index=df.index)

    ml_cols = (["id_parcelle"] if "id_parcelle" in df_export.columns else []) \
              + ["block_id"] + feat_cols + ["proxy_label", "CPI_v3"]
    ml_cols = [c for c in ml_cols if c in df_export.columns]

    df_ml = df_export[valid_labeled][ml_cols].copy() if valid_labeled.any() \
            else pd.DataFrame(columns=ml_cols)

    df_ml.to_csv(OUTPUT_CSV_ML, index=False)
    n_pos = (df_ml.get("proxy_label", pd.Series()) == 1).sum()
    n_neg = (df_ml.get("proxy_label", pd.Series()) == 0).sum()
    print(f"\n  ✓ Dataset ML : {OUTPUT_CSV_ML}")
    print(f"    {len(df_ml):,} parcelles × {len(df_ml.columns)} colonnes")
    print(f"    Labels : {n_pos} pos / {n_neg} neg")

    # README coéquipier V3
    readme = f"""# Terra-IA — Dataset ML V3 — README pour le coéquipier ML

## Fichier : {OUTPUT_CSV_ML}
Généré par terra_ia_pipeline_v3.py — Mars 2026

## Features disponibles ({len(feat_cols)})
```
{chr(10).join(f" {f:35s} [{next((g for g,fs in FEATURE_GROUPS.items() if f in fs),'?')}]" for f in feat_cols)}
```

## Colonne cible : proxy_label
  1  = constructible probable  (H1 LiDAR + H3 DVF si disponible)
  0  = contraignant             (H2 LiDAR + H4 DVF si disponible)
  -1 = ABSTAIN (exclu du dataset — non present dans ce fichier)

## Colonne CPI_v3
  Score déterministe V3 [0-100] — baseline de comparaison

## Colonne block_id
  Identifiant bloc spatial (grille {GRID_SIZE_M}m) — OBLIGATOIRE pour GroupKFold

## Modèle recommandé — XGBoost LambdaMART
```python
import xgboost as xgb
import numpy as np

# Trier par bloc pour LambdaMART
df_sorted = df.sort_values('block_id')
X = df_sorted[FEAT_COLS].values
y = df_sorted['proxy_label'].values

_, qid_inv = np.unique(df_sorted['block_id'], return_inverse=True)
_, counts  = np.unique(qid_inv, return_counts=True)
```
```

```
model = xgb.XGBRanker(
    objective='rank:ndcg',
    n_estimators=300,      # best GridSearch V3
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)
model.fit(X, y, group=counts)
```

Validation croisée — GroupKFold spatial OBLIGATOIRE
```python
from sklearn.model_selection import GroupKFold

gkf = GroupKFold(n_splits=5)
for tr_idx, val_idx in gkf.split(X, y, groups=df['block_id']):
    # Trier par bloc dans le fold d'entraînement
    blocks_tr = df['block_id'].values[tr_idx]
    _, qid_tr = np.unique(blocks_tr, return_inverse=True)
    sort_idx  = np.argsort(qid_tr)
    _, cnts   = np.unique(qid_tr[sort_idx], return_counts=True)
    model.fit(X[tr_idx][sort_idx], y[tr_idx][sort_idx], group=cnts)
    scores = model.predict(X[val_idx])
    # Métriques : NDCG@20, Precision@20, AUC-ROC
```

Métriques cibles (par ordre d'importance)
  1. NDCG@20   — qualité du ranking (principale)
  2. Precision@20 — top-k pertinence métier
  3. AUC-ROC   — discrimination générale
  4. Spearman vs CPI_v3 — cohérence avec score déterministe

SHAP — explicabilité
```python
import shap
explainer   = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X)
# shap.summary_plot(shap_values, X, feature_names=FEAT_COLS)
```

Points de vigilance (issus analyse V2/V3)
  - NE PAS utiliser surface_m2 comme feature ML (redondance avec labels)
  - Vérifier distribution classes par bloc avant CV (blocs monoclasses)
  - XGBRanker nécessite le tri par block_id avant fit()
  - NDCG = 1.000 pour RF suggère une redondance feature/label — normal si DVF absent

Références
  LambdaMART   : Chen & Guestrin (2016) XGBoost — KDD 2016
  SHAP   : Lundberg & Lee (2017) NeurIPS 30
  Spatial CV   : Valavi et al. (2019) Methods Ecol Evol 10(2):225-232
  Snorkel   : Ratner et al. (2017) VLDB Journal 26:793-817
  Burges   : Burges et al. (2006) Learning to Rank — ICML
"""
```
```
    with open("README_ML_dataset_v3.md", "w", encoding="utf-8") as f:
        f.write(readme)
    print(f"\n ✓ README coéquipier : README_ML_dataset_v3.md")

    # Rapport JSON
    valid = df[df.get("is_valid", pd.Series(True, index=df.index))]
    stats_report.update({
        "version": "3.0",
        "commune": COMMUNE_CODE,
        "n_total": int(len(df)),
        "n_valid": int(df.get("is_valid", pd.Series(True, index=df.index)).sum()),
        "n_labeled": int(len(df_ml)),
        "n_pos": int(n_pos), "n_neg": int(n_neg),
        "n_blocks": int(df.get("block_id", pd.Series()).nunique()),
        "cpi_v3_stats": {
            "mean": float(valid["CPI_v3"].mean()) if "CPI_v3" in valid else None,
            "std": float(valid["CPI_v3"].std()) if "CPI_v3" in valid else None,
            "min": float(valid["CPI_v3"].min()) if "CPI_v3" in valid else None,
```
```

```python
            "max":  float(valid["CPI_v3"].max())  if "CPI_v3" in valid else None,
        },
        "features": feat_cols,
        "all_features": ALL_FEATURES,
        "grid_size_m": GRID_SIZE_M,
        "tau_softmin":  float(TAU_SOFTMIN) if TAU_SOFTMIN else None,
    })
    with open(OUTPUT_REPORT, "w", encoding="utf-8") as f:
        json.dump(stats_report, f, indent=2, ensure_ascii=False, default=str)
    print(f"\n  ✓ Rapport stats JSON : {OUTPUT_REPORT}")

    # Résumé console
    print(f"\n  ── Distribution CPI_v3 (parcelles valides) ──")
    for lbl in ["Éliminatoire","Faible","Moyen","Bon","Excellent"]:
        n = (df["CPI_v3_label"] == lbl).sum() if "CPI_v3_label" in df.columns else 0
        p = n / len(df) * 100
        print(f"    {lbl:<15} : {n:>6,} ({p:>5.1f}%)")

    if "CPI_ML_v3" in df.columns:
        print(f"\n  ── Top 5 parcelles CPI_ML_v3 ──")
        show_cols = [c for c in ["CPI_ML_v3","CPI_v3","slope_p50","slope_p90",
                                 "svf_mean","twi_mean","surface_m2"] if c in df.columns]
        valid_sc = df[df.get("is_valid", pd.Series(True, index=df.index))]
        print(valid_sc.nlargest(5, "CPI_ML_v3")[show_cols].round(2).to_string())


# ==============================================================================
# PIPELINE PRINCIPAL V3
# ==============================================================================

def run_pipeline_v3():
    global TAU_SOFTMIN

    print("\n" + "=" * 70)
    print("  TERRA-IA — Pipeline complet v3.0 — Chambéry (73065)")
    print("  Couverture : 24 dalles IGN (4×6 km) | DVF Savoie | Blocs 300m")
    print("=" * 70)

    stats_report = {}

    # ═══════════════════════════════════════════════════════════════════
    # ÉTAPE 0 — TÉLÉCHARGEMENT
    # ═══════════════════════════════════════════════════════════════════
    print(f"\n[ÉTAPE 0-A] Téléchargement {len(URLS_MNT)} dalles MNT + "
          f"{len(URLS_MNH)} dalles MNH (IGN LiDAR HD)")
    if SKIP_DOWNLOAD:
        print("  SKIP_DOWNLOAD=True")
        mnt_paths = sorted(DATA_DIR.glob("*MNT*.tif"))
        mnh_paths = sorted(DATA_DIR.glob("*MNH*.tif"))
        print(f"  MNT trouvés : {len(mnt_paths)}  |  MNH trouvés : {len(mnh_paths)}")
    else:
        mnt_paths = download_all_dalles(URLS_MNT, "MNT")
        mnh_paths = download_all_dalles(URLS_MNH, "MNH")

    print("\n[ÉTAPE 0-B] Parcelles cadastrales (Etalab 2.0)")
    if not download_parcelles(COMMUNE_CODE, PARCELLES_PATH):
        print("  ✗ Erreur"); sys.exit(1)

    print("\n[ÉTAPE 0-C] DVF Savoie (data.gouv.fr)")
    dvf_ok = download_dvf_v3(DVF_PATH)
    stats_report["dvf_available"] = dvf_ok

    print("\n[ÉTAPE 0-D] Fusion des dalles")
    merge_dalles(mnt_paths, MNT_PATH, "MNT")
    merge_dalles(mnh_paths, MNH_PATH, "MNH")

    if not MNT_PATH.exists() or not MNH_PATH.exists():
        print("  ✗ Rasters manquants"); sys.exit(1)

    # ═══════════════════════════════════════════════════════════════════
    # ÉTAPE 1 — VALIDATION
    # ═══════════════════════════════════════════════════════════════════
    print("\n[ÉTAPE 1] Validation des données")
    if not (validate_raster(MNT_PATH, "MNT") and validate_raster(MNH_PATH, "MNH")):
```

```python
    sys.exit(1)
parcelles = load_parcelles(PARCELLES_PATH)

# ═══════════════════════════════════════════════════════════════════
# ÉTAPE 2 — CALCUL DES FEATURES RASTER
# ═══════════════════════════════════════════════════════════════════
print("\n[ÉTAPE 2] Calcul des features raster (9 features)")

with rasterio.open(MNT_PATH) as src:
    mnt_data = src.read(1).astype(float)
    cellsize = src.res[0]
    nd       = src.nodata or -9999.0
mnt_data[mnt_data == nd] = np.nan

# Slope Horn (1981)
slope_path = RASTER_DIR / "slope_v3.tif"
if not slope_path.exists() or not SKIP_FEATURES:
    print("  ⚙  Slope Horn (1981) ...", end=" ", flush=True)
    slope = compute_slope_raster(mnt_data, cellsize)
    save_tif(slope, MNT_PATH, slope_path)
    print(f"OK  moy={np.nanmean(slope):.2f}°")
else:
    print(f"  ✓ Slope existant : {slope_path.name}")

# TRI Riley (1999)
tri_path = RASTER_DIR / "tri_v3.tif"
if not tri_path.exists() or not SKIP_FEATURES:
    print("  ⚙  TRI Riley (1999) ...", end=" ", flush=True)
    tri = compute_tri_riley(mnt_data)
    save_tif(tri, MNT_PATH, tri_path)
    print(f"OK  moy={np.nanmean(tri):.4f}m")
else:
    print(f"  ✓ TRI existant")

# TWI + Thalweg (Beven & Kirkby 1979 / Barnes 2014) — un seul passage D8
twi_path     = RASTER_DIR / "twi_v3.tif"
thalweg_path = RASTER_DIR / "thalweg_v3.tif"
if not twi_path.exists() or not SKIP_FEATURES:
    print("  ⚙  TWI_urban + Thalweg (Beven & Kirkby 1979) ...", end=" ", flush=True)
    twi_r, thalweg_r = compute_twi_and_thalweg(MNT_PATH, BETA_MIN_DEG)
    if twi_r is not None:
        save_tif(twi_r, MNT_PATH, twi_path)
        save_tif(thalweg_r, MNT_PATH, thalweg_path)
    else:
        twi_path = thalweg_path = None
else:
    print(f"  ✓ TWI/Thalweg existants")
    if not twi_path.exists():     twi_path     = None
    if not thalweg_path.exists(): thalweg_path = None

# SVF (Zakšek 2011)
svf_path = RASTER_DIR / "svf_v3.tif"
if not svf_path.exists() or not SKIP_FEATURES:
    print("  ⚙  Sky View Factor (Zakšek 2011) ...", end=" ", flush=True)
    svf_r = compute_svf(MNH_PATH)
    if svf_r is not None:
        save_tif(svf_r, MNH_PATH, svf_path)
    else:
        svf_path = None
else:
    print(f"  ✓ SVF existant")

# Hillshade hiver
hs_path = RASTER_DIR / "hillshade_hiver_v3.tif"
if not hs_path.exists() or not SKIP_FEATURES:
    print("  ⚙  Hillshade solstice hiver (21 déc. 12h) ...", end=" ", flush=True)
    hs_r = compute_hillshade_winter(mnt_data, cellsize, LAT_DEG)
    save_tif(hs_r, MNT_PATH, hs_path)
    print(f"OK  moy={np.nanmean(hs_r):.3f}")
else:
    print(f"  ✓ Hillshade existant")

# Aspect orientation sud
asp_path = RASTER_DIR / "aspect_south_v3.tif"
```

```python
if not asp_path.exists() or not SKIP_FEATURES:
    print("  ⚙  Aspect orientation sud ...", end=" ", flush=True)
    asp_r = compute_aspect_south(mnt_data, cellsize)
    save_tif(asp_r, MNT_PATH, asp_path)
    print(f"OK  ratio_sud={np.nanmean(asp_r):.3f}")
else:
    print(f"  ✓ Aspect existant")

# Profile curvature (Zevenbergen & Thorne 1987)
curv_path = RASTER_DIR / "curvature_v3.tif"
if not curv_path.exists() or not SKIP_FEATURES:
    print("  ⚙  Profile curvature (Zevenbergen & Thorne 1987) ...", end=" ", flush=True)
    curv_r = compute_profile_curvature(mnt_data, cellsize)
    save_tif(curv_r, MNT_PATH, curv_path)
    print(f"OK  moy={np.nanmean(curv_r):.6f}")
else:
    print(f"  ✓ Curvature existante")

# ═══════════════════════════════════════════════════════════════════
# ÉTAPE 3 — ZONAL STATS PAR PARCELLE
# ═══════════════════════════════════════════════════════════════════
print("\n[ÉTAPE 3] Extraction features par parcelle (zonal stats)")
features = {}

print("  ⚙  Slope P50/P90/std ...", end=" ", flush=True)
features.update(zonal_percentile(slope_path, parcelles, "slope", [50, 90]))
print("OK")

print("  ⚙  TRI ...", end=" ", flush=True)
features.update(zonal(tri_path, parcelles, "tri", ["mean", "max"]))
print("OK")

if twi_path and Path(twi_path).exists():
    print("  ⚙  TWI ...", end=" ", flush=True)
    features.update(zonal(twi_path, parcelles, "twi", ["mean", "max"]))
    print("OK")

if thalweg_path and Path(thalweg_path).exists():
    print("  ⚙  Thalweg ratio ...", end=" ", flush=True)
    features.update(zonal(thalweg_path, parcelles, "has_thalweg", ["mean"]))
    print("OK")

if svf_path and Path(svf_path).exists():
    print("  ⚙  SVF ...", end=" ", flush=True)
    features.update(zonal(svf_path, parcelles, "svf", ["mean", "min"]))
    print("OK")

print("  ⚙  Hillshade hiver ...", end=" ", flush=True)
features.update(zonal(hs_path, parcelles, "hillshade_winter", ["mean"]))
print("OK")

print("  ⚙  Aspect sud ...", end=" ", flush=True)
features.update(zonal(asp_path, parcelles, "aspect_south_ratio", ["mean"]))
print("OK")

print("  ⚙  Profile curvature ...", end=" ", flush=True)
features.update(zonal(curv_path, parcelles, "profile_curvature", ["mean"]))
print("OK")

print("  ⚙  Hauteur objets voisins (MNH buffer 20m) ...", end=" ", flush=True)
buf = parcelles.copy()
buf["geometry"] = parcelles.geometry.buffer(20)
buf_path = DATA_DIR / "buf20_v3.geojson"
buf.to_file(buf_path, driver="GeoJSON")
features.update(zonal(MNH_PATH, buf, "height_obj", ["max", "mean"]))
print("OK")

try:
    import osmnx as ox
    print("  ⚙  Distance route (OSM) ...", end=" ", flush=True)
    G     = ox.graph_from_place("Chambéry, France", network_type="drive")
    roads = ox.graph_to_gdfs(G, nodes=False).to_crs("EPSG:2154").geometry.unary_union
    features["dist_road_m"] = parcelles.geometry.centroid.apply(
        lambda p: p.distance(roads)).values
```

```
        print(f"OK  médiane={np.nanmedian(features['dist_road_m']):.1f}m")
    except ImportError:
        print("  ⚠  osmnx non installé (pip install osmnx)")
        features["dist_road_m"] = [np.nan] * len(parcelles)

    # ── DataFrame principal
    df = pd.DataFrame({"commune": COMMUNE_CODE}, index=range(len(parcelles)))
    if "id" in parcelles.columns:
        df["id"] = parcelles["id"].values
    for col, vals in features.items():
        if vals is not None and len(vals) == len(df):
            df[col] = vals

    # Blocs spatiaux V3 (300m)
    df["block_id"] = create_spatial_blocks(parcelles, GRID_SIZE_M).values
    print(f"\n  Blocs spatiaux {GRID_SIZE_M}m : {df['block_id'].nunique()} blocs")

    # ═══════════════════════════════════════════════════════════════════
    # ÉTAPE 4 — FILTRAGE
    # ═══════════════════════════════════════════════════════════════════
    print("\n[ÉTAPE 4] Filtrage et nettoyage")
    df = filter_parcelles(df, parcelles)
    stats_report["filtrage"] = {
        "n_total": int(len(df)),
        "n_valid": int(df["is_valid"].sum()),
        "n_nan":   int((df["nan_ratio"] > SEUIL_NAN).sum()),
    }

    # ═══════════════════════════════════════════════════════════════════
    # ÉTAPE 5 — SCORE CPI V3
    # ═══════════════════════════════════════════════════════════════════
    print("\n[ÉTAPE 5] Score CPI V3 — calibration τ + score déterministe")

    # Calibration automatique τ
    valid_df = df[df["is_valid"]]
    group_cols = ["score_pente","score_hydro","score_morpho","score_soleil"]

    # Pré-calcul des scores groupes pour calibrer τ
    def _pre_group_scores(d):
        from copy import deepcopy
        d2 = deepcopy(d)
        def get(col, def_=0.5):
            if col in d2.columns:
                return d2[col].fillna(d2[col].median() if d2[col].notna().any() else def_)
            return pd.Series(def_, index=d2.index)
        sp = (0.40*normalize(get("slope_p50"),invert=True)
             +0.35*normalize(get("slope_p90"),invert=True)
             +0.25*normalize(get("slope_std"),invert=True))
        sh = normalize(get("twi_mean"),invert=True)
        sm = (0.55*normalize(get("profile_curvature_mean"),invert=False)
             +0.45*normalize(get("tri_mean"),invert=True))
        se = (0.40*normalize(get("aspect_south_ratio_mean"))
             +0.35*normalize(get("hillshade_winter_mean"))
             +0.25*normalize(get("svf_mean")))
        return pd.DataFrame({
            "p": sp*100, "h": sh*100, "m": sm*100, "s": se*100
        })

    if len(valid_df) > 50:
        gs = _pre_group_scores(valid_df)
        TAU_SOFTMIN = calibrate_tau(gs, target_std=18.0)
    else:
        TAU_SOFTMIN = 10.0
        print(f"    τ par défaut : {TAU_SOFTMIN}")

    df = compute_cpi_v3(df, TAU_SOFTMIN)

    # ═══════════════════════════════════════════════════════════════════
    # ÉTAPE 6 — LABELS SNORKEL V3
    # ═══════════════════════════════════════════════════════════════════
    print("\n[ÉTAPE 6] Labels Snorkel V3 (weak supervision + DVF)")
    dvf_data = prepare_dvf(DVF_PATH) if dvf_ok else None
    df["proxy_label"] = create_snorkel_labels_v3(df, dvf_data)
```

```python
    # ======================================================================
    # ÉTAPE 7 — ANALYSE DES BLOCS
    # ======================================================================
    print("\n[ÉTAPE 7] Analyse équilibre classes par bloc spatial")
    bloc_stats = analyze_blocks(df, df["proxy_label"])
    stats_report["bloc_analysis"] = bloc_stats

    # ======================================================================
    # ÉTAPE 8 — COMPARAISON MODÈLES
    # ======================================================================
    print("\n[ÉTAPE 8] Comparaison modèles ML (GroupKFold + GridSearch XGBoost)")
    ml_results = compare_models_v3(df, df["proxy_label"], parcelles)
    stats_report["ml_results"] = {
        k: {kk: vv for kk, vv in v.items() if not kk.startswith("_")}
        for k, v in ml_results.items() if isinstance(v, dict)
    }

    # ======================================================================
    # ÉTAPE 9 — LAMBDAMART FINAL + SHAP
    # ======================================================================
    print("\n[ÉTAPE 9] XGBoost LambdaMART final + SHAP V3")
    # Récupérer les meilleurs params du GridSearch
    best_params = None
    for name, r in ml_results.items():
        if "rank:ndcg" in name or "XGBoost" in name:
            if isinstance(r, dict) and "_best_params" in r:
                best_params = r["_best_params"]
    train_and_explain_v3(df, df["proxy_label"], best_params)

    # ======================================================================
    # ÉTAPE 10 — EXPORT
    # ======================================================================
    print("\n[ÉTAPE 10] Export des résultats")
    export_v3(df, parcelles, stats_report)

    print("\n" + "=" * 70)
    print("  ✓ PIPELINE V3 TERMINÉ")
    print(f"  → Dataset complet   : {OUTPUT_CSV_V3}")
    print(f"  → Dataset ML        : {OUTPUT_CSV_ML}")
    print(f"  → README coéquipier : README_ML_dataset_v3.md")
    print(f"  → SHAP features     : {DATA_DIR}/shap_importance_v3.csv")
    print(f"  → SHAP groupes      : {DATA_DIR}/shap_importance_groupes_v3.csv")
    print(f"  → Rapport JSON      : {OUTPUT_REPORT}")
    print("=" * 70 + "\n")

    return df


if __name__ == "__main__":
    run_pipeline_v3()
```